SISTEMAS EMBEBIDOS

Universidad Politécnica del

Bicentenario.

Práctica 6: Sistema de Control de Temperatura ON-OFF con Histéresis Basado en ESP32

Ingeniería Mecatronica 701

Edgar Uriel Rodriguez Moreno *24033660*

Jorge Zamorano Chavez *24033247*

Axel Gabriel Domínguez Macias *24032952*

Braulio Vázquez Morado *24032975*

Daisy Jackeline Hernandez Ramos *24033069*

Silao, Guanajuato a 6 de Noviembre de 2026.

Introducción

El control automático de temperatura constituye uno de los pilares fundamentales dentro de la ingeniería en sistemas embebidos, la automatización industrial y el desarrollo de instrumentación electrónica. En numerosos procesos domésticos, comerciales e industriales tales como incubadoras, hornos de secado, sistemas de climatización, cámaras frigoríficas o reactores químicos mantener una variable térmica dentro de un rango específico resulta indispensable para garantizar la eficiencia operativa, la seguridad de los equipos y la calidad del producto final. Para resolver esta necesidad sin recurrir a arquitecturas excesivamente complejas o costosas, el controlador todo-nada con banda de histéresis se posiciona como una estrategia robusta, confiable y altamente eficiente.

El presente reporte documenta el diseño, la implementación física, la programación y la validación experimental de un sistema embebido de regulación térmica bidireccional gobernado por un microcontrolador ESP32. A diferencia de los sistemas de lazo abierto o de aquellos cuyos parámetros se encuentran fijos dentro del código fuente, este prototipo incorpora una interfaz hombre-máquina local compuesta por una pantalla de cristal líquido de 16×2 con adaptador I2C, un teclado matricial de membrana de 4×4 y un pulsador físico de selección. Esta arquitectura permite al usuario visualizar en tiempo real la temperatura interna de la cámara térmica, identificar el estado activo del ciclo y reconfigurar dinámicamente los umbrales mínimo y máximo de operación sin necesidad de recompilar ni volver a cargar el firmware. Asimismo, el sistema integra un sensor digital de temperatura DS18B20 de alta precisión y una etapa de aislamiento electromecánico mediante un módulo de relevadores de dos canales, encargado de conmutar de manera segura tanto un elemento calefactor como un sistema de enfriamiento termoeléctrico. El desarrollo del software se llevó a cabo bajo el entorno PlatformIO utilizando el framework de Arduino en lenguaje C++, logrando una ejecución no bloqueante mediante temporización por software y validando satisfactoriamente cada uno de los requerimientos planteados en la práctica.

Marco Teórico

El funcionamiento integral del prototipo se sustenta en la interacción coordinada de diversos componentes electrónicos, transductores, actuadores y protocolos de comunicación digital, los cuales se detallan a continuación junto con su principio operativo y compatibilidad de programación.

El *microcontrolador ESP32* constituye la unidad central de procesamiento y control del sistema. Se trata de un System-on-Chip de 32 bits diseñado por Espressif Systems, basado en una arquitectura Xtensa LX6 de doble núcleo capaz de operar a frecuencias de hasta 240 MHz, con conectividad inalámbrica Wi-Fi y Bluetooth integrada, memoria SRAM interna y múltiples periféricos de entrada y salida de propósito general, conversores analógico-digitales, temporizadores y controladores de bus serie como I2C, SPI y UART. En esta práctica, el ESP32 se encarga de gestionar la lectura periódica del sensor térmico, decodificar las pulsaciones del teclado matricial y del botón selector, ejecutar el algoritmo de control por histéresis, actualizar la pantalla LCD y enviar las señales lógicas de activación hacia la etapa de potencia. Respecto a su ecosistema de desarrollo, el ESP32 es ampliamente compatible con C y C++ a través del framework oficial ESP-IDF y del framework Arduino en entornos como PlatformIO o Arduino IDE, así como con MicroPython, CircuitPython, Lua, Rust y JavaScript.

El *sensor digital de temperatura DS18B20* es un transductor semiconductor fabricado por Maxim Integrated, actualmente Analog Devices, diseñado para medir temperaturas en un rango de -55 °C a +125 °C, con una precisión de ±0.5 °C en el intervalo de -10 °C a +85 °C y una resolución configurable entre 9 y 12 bits. Su principal ventaja frente a sensores analógicos como el LM35 o los termistores radica en que realiza la conversión analógico-digital internamente y transmite el dato en formato digital a través del protocolo propietario 1-Wire, lo cual elimina errores por caída de tensión o ruido eléctrico en cables largos y permite conectar múltiples sensores sobre una misma línea de datos utilizando únicamente una resistencia pull-up externa de 4.7 kΩ. En cuanto a su programación, el DS18B20 es compatible

con prácticamente cualquier lenguaje utilizado en sistemas embebidos que permita manipular tiempos de pulso en pines digitales, destacando C y C++ mediante las bibliotecas OneWire y DallasTemperature en Arduino y PlatformIO, MicroPython con los módulos ds18x20 y onewire, Python en computadoras de placa única como Raspberry Pi, C en microcontroladores PIC o AVR, y Rust.

La *pantalla LCD alfanumérica de 16×2 con módulo adaptador I2C* funciona como el dispositivo principal de salida visual para el usuario. El display LCD estándar basado en el controlador HD44780 permite representar dos filas de dieciséis caracteres cada una, pero su interfaz paralela nativa exige al menos seis pines de control y datos del microcontrolador. Al acoplar la mochila o módulo adaptador con el expansor de entradas y salidas PCF8574, la comunicación se reduce a un bus serie síncrono I2C de apenas dos líneas, SDA para datos y SCL para reloj, además de incorporar un potenciómetro para el ajuste de contraste y un puente para controlar la retroiluminación. Este componente es compatible con C y C++ utilizando librerías como LiquidCrystal_I2C en Arduino y PlatformIO o controladores I2C nativos en ESP-IDF y STM32 HAL, MicroPython, Python, CircuitPython y ensamblador.

El *teclado matricial de membrana 4×4* es un dispositivo de entrada compuesto por dieciséis pulsadores distribuidos en una matriz de cuatro filas y cuatro columnas. Internamente, cada tecla funciona como un interruptor normalmente abierto que interconecta una línea de fila específica con una línea de columna al ser presionada. Mediante una técnica de barrido secuencial o multiplexación, el microcontrolador envía pulsos lógicos a través de las filas y lee el estado de las columnas para determinar exactamente qué tecla fue accionada, ahorrando ocho pines en comparación con el uso de botones individuales. En el prototipo, el teclado permite ingresar dígitos numéricos del 0 al 9 para definir los nuevos valores de temperatura mínima y máxima, confirmar el dato con la tecla # o limpiar la entrada con la tecla *. Su lectura es compatible con C y C++ empleando la biblioteca Keypad en Arduino y PlatformIO o rutinas matriciales en ESP-IDF, PIC y AVR, MicroPython, CircuitPython, Python y Rust.

El *pulsador táctil* conectado en configuración *pull-up* interna actúa como un selector discreto de modo de edición. Al cerrar el circuito hacia tierra cuando es presionado, genera un flanco de bajada en el pin GPIO 25 del ESP32, permitiendo alternar de forma intuitiva entre la edición del límite inferior de temperatura Min y el límite superior Max, aplicando un filtro antirrebote por software para evitar lecturas erróneas causadas por la vibración mecánica de sus contactos. Por su naturaleza electromecánica elemental, es compatible con absolutamente todos los lenguajes de programación capaces de leer entradas digitales, incluyendo C, C++, Python, MicroPython, Rust, Lua y lenguaje ensamblador.

El *módulo de relevadores de 2 canales* conforma la etapa de aislamiento y conmutación de potencia del sistema. Dado que los pines GPIO del ESP32 operan a un nivel lógico de 3.3 V y entregan una corriente máxima muy reducida de alrededor de 12 a 40 mA, resulta imposible alimentar directamente cargas de alta potencia desde el microcontrolador. Este módulo integra optoacopladores y transistores de excitación que protegen al ESP32 frente a picos de voltaje inverso generados por las bobinas electromagnéticas de los relevadores. Cuando el ESP32 envía un estado lógico bajo LOW, la bobina del relevador correspondiente se energiza y conmuta sus contactos mecánicos internos común, normalmente abierto y normalmente cerrado, permitiendo encender o apagar cargas externas de corriente alterna o corriente directa de mayor amperaje. Su control mediante señales digitales binarias es compatible con C, C++, MicroPython, Python, Rust, Lua y cualquier entorno embebido.

La *celda Peltier acoplada a un disipador de aluminio y ventilador* cumple la función de actuador de enfriamiento dentro de la cámara térmica. Basada en el efecto Peltier, esta placa semiconductora genera una transferencia de calor desde una de sus caras hacia la opuesta cuando circula una corriente continua a través de sus uniones tipo P y tipo N; de este modo, la cara fría extrae energía térmica del interior de la caja, mientras que el disipador aletado y el ventilador evacúan el calor acumulado en la cara caliente hacia el exterior para evitar el sobrecalentamiento del dispositivo. Al ser un actuador pasivo de potencia gobernado a través del relevador,

su control lógico es compatible con cualquier lenguaje de programación empleado en el microcontrolador anfitrión como C, C++, MicroPython o Python.

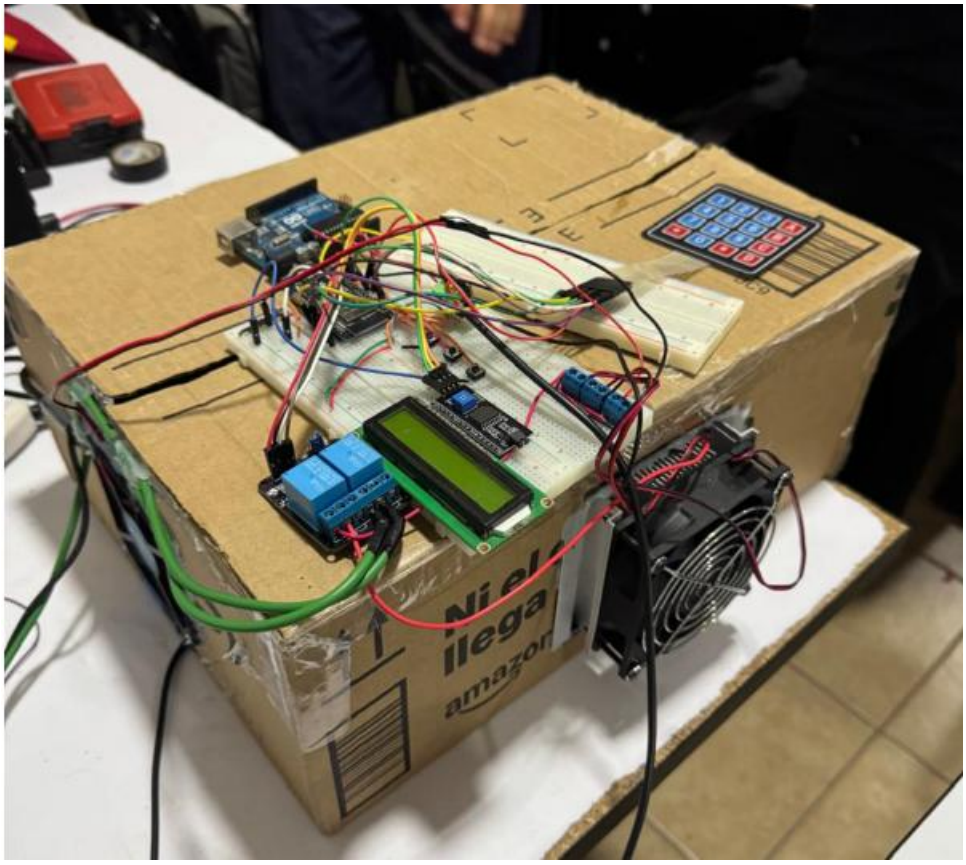
El *elemento calefactor* opera como el actuador encargado de elevar la temperatura dentro del recinto cerrado. Al circular corriente eléctrica a través de su filamento resistivo por efecto Joule, convierte la energía eléctrica en radiación térmica y calor convectivo, incrementando rápidamente la temperatura registrada por el sensor DS18B20 durante la fase de calentamiento. Al igual que la celda Peltier, su activación se gestiona de manera binaria mediante el segundo canal del módulo de relevadores, haciéndolo compatible con cualquier lenguaje de programación de sistemas embebidos como C, C++, MicroPython o Rust.

Las *protoboards, cables jumper DuPont, borneras de conexión y fuente de alimentación externa* brindan el soporte físico, la interconexión eléctrica y el suministro energético requerido tanto para la etapa lógica de 3.3 V y 5 V como para la etapa de potencia de los actuadores y relevadores, asegurando estabilidad en las referencias de tierra común y facilidad para realizar mediciones o ajustes durante las pruebas experimentales.

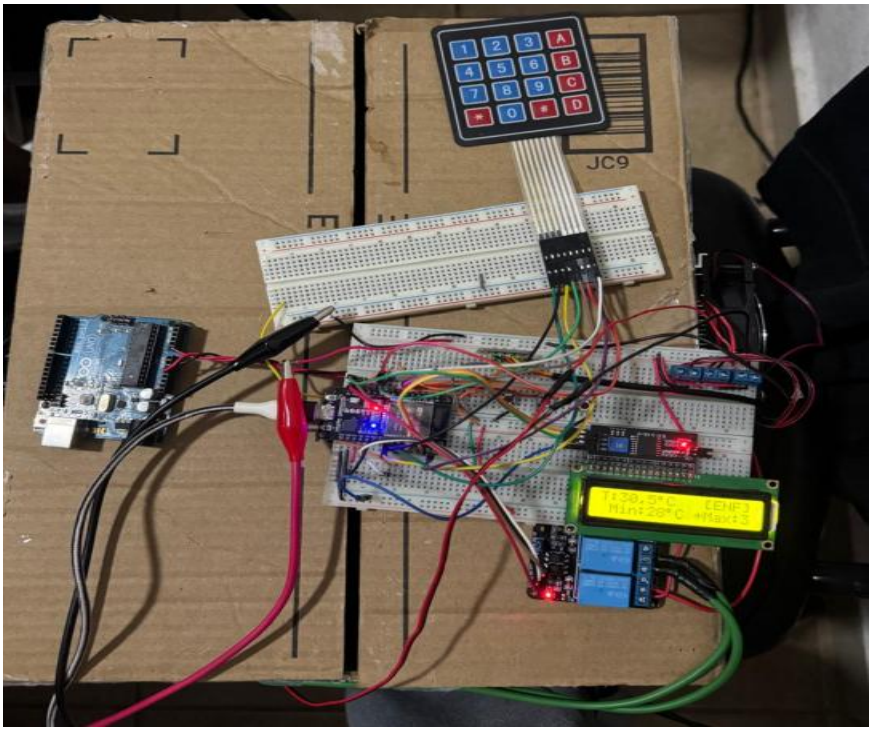
Desarrollo

La ejecución del proyecto comenzó con la adquisición y verificación individual de cada uno de los componentes electrónicos necesarios para construir tanto el sistema de control como la cámara térmica de pruebas. Una vez reunido el material incluyendo la tarjeta ESP32 DevKit V1, el sensor encapsulado DS18B20 con su resistencia pull-up de 4.7 kΩ, la pantalla LCD 16×2 con interfaz I2C, el teclado matricial 4×4, el módulo de relevadores de dos canales, el ensamblaje de celda Peltier con disipador y ventilador, el foco calefactor, protoboards y fuentes de alimentación se procedió al ensamblaje físico del prototipo sobre una caja de cartón acondicionada como recinto térmico aislado.

En una de las paredes laterales de la cámara se realizó una abertura exacta para empotrar el sistema de enfriamiento termoeléctrico, dejando la cara fría de la celda Peltier orientada hacia el interior de la caja y el disipador de aluminio junto con el ventilador de extracción hacia el exterior, asegurando así una correcta disipación térmica. En el interior del recinto se instalaron el elemento calefactor y el sensor DS18B20 en una posición estratégica que permitiera medir de forma representativa la temperatura del aire interno sin recibir radiación directa excesiva de la lámpara. Sobre la parte superior de la caja se montaron dos protoboards para distribuir ordenadamente el cableado: en ellas se alojó el ESP32, los pulsadores de selección y reinicio, la pantalla LCD I2C, el módulo de dos relevadores y los conectores del teclado matricial 4×4 adherido a un costado para facilitar su manipulación por parte del usuario.



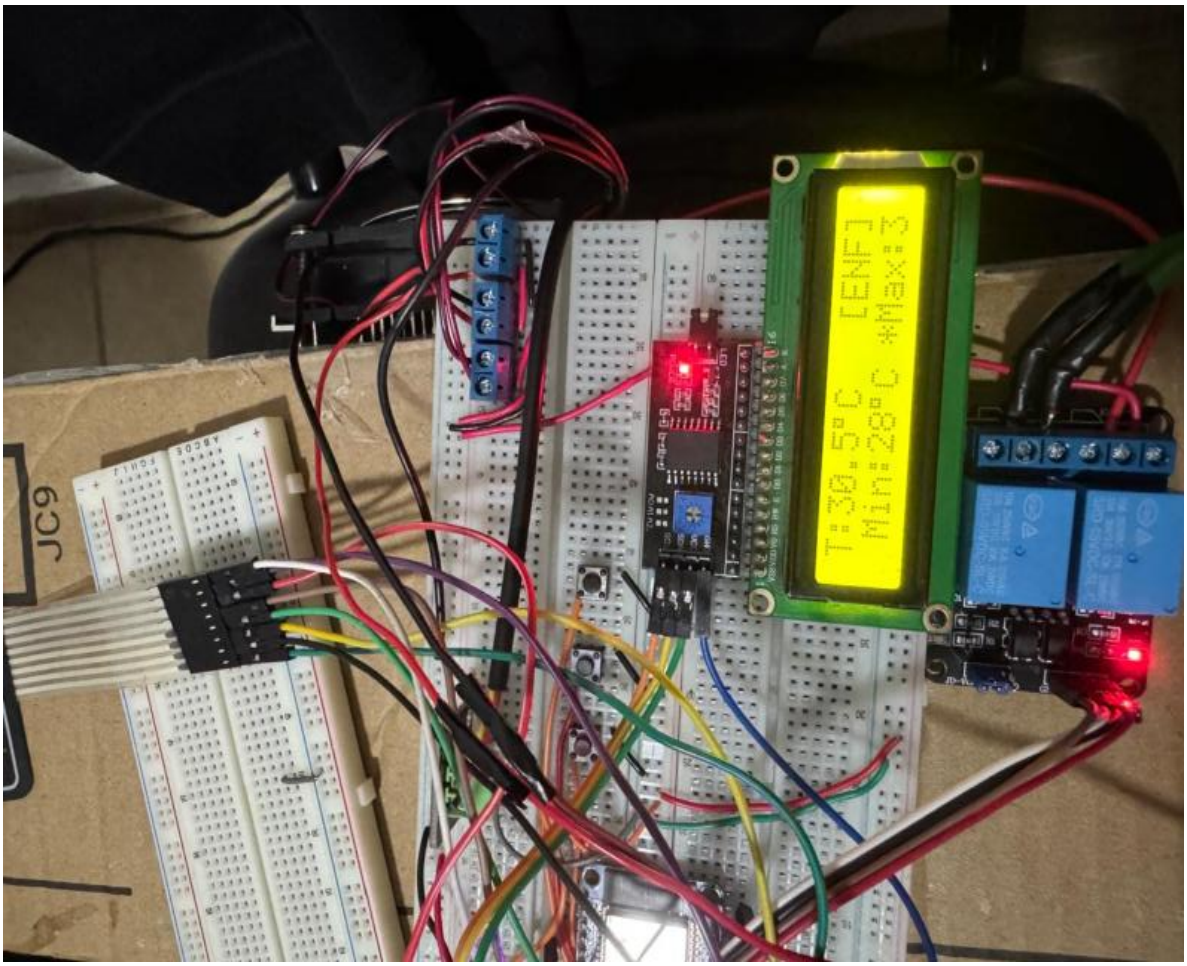
Posteriormente, se realizó la interconexión eléctrica cuidando la separación entre las líneas de control de bajo voltaje y las líneas de alimentación de potencia de los actuadores. La pantalla LCD I2C se conectó al bus I2C predeterminado del ESP32 mediante los pines GPIO 21 para SDA y GPIO 22 para SCL, configurando su dirección hexadecimal en 0x27. El sensor DS18B20 se enlazó al pin GPIO 4 con su respectiva resistencia pull-up de 4.7 kΩ entre la línea de datos y 3.3 V. Para el teclado matricial 4×4 se asignaron los pines GPIO 19, 18, 5 y 27 a las cuatro filas, y los pines GPIO 32, 14, 23 y 26 a las cuatro columnas, evitando el uso de pines reservados o conflictivos durante el arranque del microcontrolador. El pulsador selector de límite entre Min y Max se conectó al pin GPIO 25 utilizando la resistencia *pull-up* interna del microcontrolador, mientras que las señales de control hacia el módulo de relevadores de lógica activa en bajo se conectaron al GPIO 2 para el relevador del foco calefactor y al GPIO 15 para el relevador de la celda Peltier.



Con el hardware completamente cableado y verificado para descartar cortocircuitos o falsos contactos, se procedió al desarrollo del firmware en Visual Studio Code mediante la extensión PlatformIO bajo el framework de Arduino en C++. Se

configuró el archivo platformio.ini incluyendo las librerías OneWire, DallasTemperature, LiquidCrystal_I2C y Keypad. La arquitectura del código se diseñó bajo un enfoque no bloqueante utilizando la función millis, evitando el uso de retardos bloqueantes dentro del bucle principal para garantizar que el teclado matricial y el botón selector respondieran de forma inmediata en cualquier instante, independientemente del intervalo de muestreo de 1 segundo establecido para la lectura del sensor DS18B20.

Dentro de la lógica de control implementada en la función controlarCiclo, se programó una máquina de dos estados, CALENTANDO y ENFRIANDO, gobernada por histéresis entre el umbral mínimo tempMin, iniciado en 30 °C, y el umbral máximo tempMax, iniciado en 60 °C. Cuando el sistema se encuentra en modo CALENTANDO, mantiene activado el relevador del foco en estado bajo y apagado el de la celda Peltier en estado alto hasta que la temperatura medida iguala o supera tempMax; en ese instante, el sistema conmuta al estado ENFRIANDO, apaga el foco y enciende la celda Peltier con su ventilador para extraer calor hasta que la temperatura desciende a tempMin o menos, momento en el cual reinicia el ciclo de calentamiento. Asimismo, se programó la rutina actualizarLCD para desplegar en la primera línea la temperatura actual con un decimal junto al indicador de estado [CAL] o [ENF], y en la segunda línea los valores vigentes de Min y Max señalando con un asterisco * cuál de los dos parámetros se encuentra seleccionado para edición mediante el botón del GPIO 25, permitiendo digitar el nuevo valor en el teclado 4×4, borrarlo con * o confirmarlo con # previa validación lógica para asegurar que tempMin sea menor que tempMax.



Finalmente, tras compilar y cargar el programa en la tarjeta ESP32, se ejecutaron las pruebas funcionales integrales frente al docente para corroborar el desempeño del sistema en condiciones reales de operación.

```
#include <Arduino.h>
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>

// ==========================================
// ASIGNACIÓN DE PINES
// ==========================================
#define DS18B20_PIN  4    // Sensor DS18B20 (GPIO 4)
#define BTN_SEL_PIN  25   // Botón para conmutar entre Min y Max
(GPIO 25)

#define RELE_FOCO    2    // Relé Foco / Calentador (GPIO 2 -
D2)
```

```
#define RELE_PELTIER 15   // Relé Peltier / Enfriador (GPIO 15 - D15)

#define LCD_SDA      21   // LCD I2C SDA
#define LCD_SCL      22   // LCD I2C SCL
#define LCD_ADDR     0x27 // Dirección I2C LCD

// Módulo de Relés Active LOW
#define RELE_ON      LOW
#define RELE_OFF     HIGH

// ==========================================
// CONFIGURACIÓN DEL TECLADO MATRICIAL 4x4 (Pines sin 16 ni 17)
// ==========================================
const byte FILAS = 4;
const byte COLUMNAS = 4;

char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

byte pinesFilas[FILAS]       = {19, 18, 5, 27}; // F1, F2, F3, F4 (Usa GPIO 27)
byte pinesColumnas[COLUMNAS] = {32, 14, 23, 26}; // C1, C2, C3, C4 (Usa GPIO 32)

Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas, pinesColumnas, FILAS, COLUMNAS);

// ==========================================
// OBJETOS Y VARIABLES DE ESTADO
// ==========================================
OneWire oneWire(DS18B20_PIN);
DallasTemperature sensors(&oneWire);
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);

float temperaturaActual = 0.0;
int tempMin = 30; // Umbral Mínimo por defecto
int tempMax = 60; // Umbral Máximo por defecto

// 0 = Editando Temp Mínima, 1 = Editando Temp Máxima
uint8_t limiteSeleccionado = 0;

String bufferTeclado = ""; // Almacena los dígitos ingresados

enum EstadoCiclo { CALENTANDO, ENFRIANDO };
EstadoCiclo estadoCiclo = CALENTANDO;

// Control de tiempos y antirrebote
unsigned long ultimaLecturaTemp = 0;
const unsigned long INTERVALO_TEMP = 1000; // Lectura cada 1 segundo

bool estadoAnteriorBtn = HIGH;
unsigned long ultimoTiempoBoton = 0;
const unsigned long DEBOUNCE_MS = 180;
```

```
// Lógica de histéresis del horno
void controlarCiclo() {
  if (estadoCiclo == CALENTANDO) {
    if (temperaturaActual >= tempMax) {
      estadoCiclo = ENFRIANDO;
    }
  } else if (estadoCiclo == ENFRIANDO) {
    if (temperaturaActual <= tempMin) {
      estadoCiclo = CALENTANDO;
    }
  }

  if (estadoCiclo == CALENTANDO) {
    digitalWrite(RELE_FOCO, RELE_ON);
    digitalWrite(RELE_PELTIER, RELE_OFF);
  } else {
    digitalWrite(RELE_FOCO, RELE_OFF);
    digitalWrite(RELE_PELTIER, RELE_ON);
  }
}

// Actualización del display LCD
void actualizarLCD() {
  char linea0[17];
  char linea1[17];

  // Línea 0: Temperatura DS18B20 y estado [CAL/ENF]
  if (temperaturaActual < -50) {
    snprintf(linea0, sizeof(linea0), "T: Error   [%s]",
             estadoCiclo == CALENTANDO ? "CAL" : "ENF");
  } else {
    snprintf(linea0, sizeof(linea0), "T:%4.1f%cC   [%s]",
             temperaturaActual, (char)223, estadoCiclo ==
CALENTANDO ? "CAL" : "ENF");
  }

  // Línea 1: Umbrales o dígitos marcados
  if (bufferTeclado.length() > 0) {
    snprintf(linea1, sizeof(linea1), "Ingresando: %-4s",
bufferTeclado.c_str());
  } else {
    snprintf(linea1, sizeof(linea1), "%cMin:%2d%cC
%cMax:%2d%cC",
             limiteSeleccionado == 0 ? '*' : ' ', tempMin,
(char)223,
             limiteSeleccionado == 1 ? '*' : ' ', tempMax,
(char)223);
  }

  lcd.setCursor(0, 0);
  lcd.print(linea0);
  lcd.setCursor(0, 1);
  lcd.print(linea1);
}

void setup() {
  Serial.begin(115200);
```

```
  pinMode(BTN_SEL_PIN, INPUT_PULLUP);

  pinMode(RELE_FOCO, OUTPUT);
  pinMode(RELE_PELTIER, OUTPUT);
  digitalWrite(RELE_FOCO, RELE_OFF);
  digitalWrite(RELE_PELTIER, RELE_OFF);

  Wire.begin(LCD_SDA, LCD_SCL);
  lcd.init();
  lcd.backlight();

  // Inicialización DS18B20
  sensors.begin();
  sensors.setResolution(9);
  sensors.setWaitForConversion(false);
  sensors.requestTemperatures();

  lcd.setCursor(0, 0);
  lcd.print(" Horno DS18B20  ");
  lcd.setCursor(0, 1);
  lcd.print(" Keypad 4x4     ");
  delay(1500);
  lcd.clear();
}

void loop() {
  unsigned long ahora = millis();

  // 1. BOTÓN SELECTOR (GPIO 25)
  bool btnActual = digitalRead(BTN_SEL_PIN);
  if (ahora - ultimoTiempoBoton >= DEBOUNCE_MS) {
    if (btnActual == LOW && estadoAnteriorBtn == HIGH) {
      limiteSeleccionado = (limiteSeleccionado == 0) ? 1 : 0;
      bufferTeclado = "";
      ultimoTiempoBoton = ahora;
    }
  }
  estadoAnteriorBtn = btnActual;

  // 2. TECLADO MATRICIAL
  char tecla = teclado.getKey();
  if (tecla != NO_KEY) {
    if (tecla >= '0' && tecla <= '9') {
      if (bufferTeclado.length() < 3) {
        bufferTeclado += tecla;
      }
    }
    else if (tecla == '#') { // Confirmar e ingresar el valor
      if (bufferTeclado.length() > 0) {
        int valorIngresado = bufferTeclado.toInt();

        if (limiteSeleccionado == 0) {
          if (valorIngresado < tempMax) tempMin =
valorIngresado;
        } else {
          if (valorIngresado > tempMin) tempMax =
valorIngresado;
        }
        bufferTeclado = "";
```

```
      }
    }
    else if (tecla == '*') { // Borrar número
      bufferTeclado = "";
    }
    actualizarLCD();
  }

  // 3. LECTURA DEL SENSOR Y CONTROL
  if (ahora - ultimaLecturaTemp >= INTERVALO_TEMP) {
    float tempRead = sensors.getTempCByIndex(0);
    if (tempRead > -55.0 && tempRead < 125.0) {
      temperaturaActual = tempRead;
    }
    sensors.requestTemperatures();
    ultimaLecturaTemp = ahora;

    controlarCiclo();
    actualizarLCD();
  }

}
```

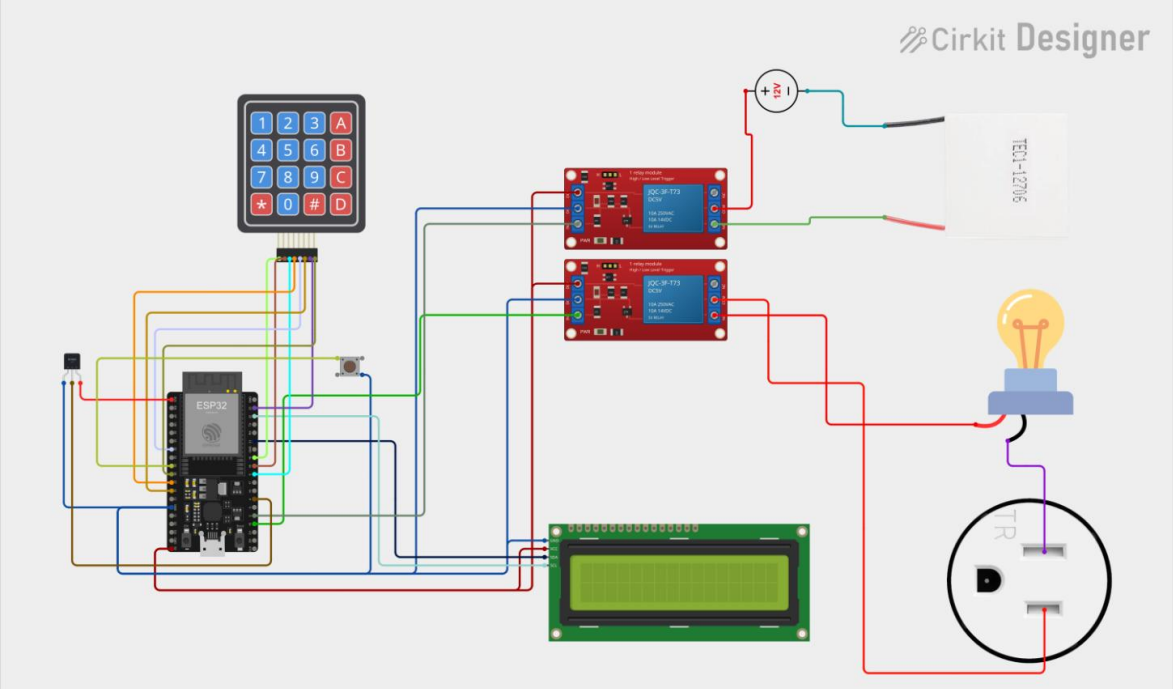


Diagrama Repositorio en Github

Resultados

Los resultados obtenidos durante la fase de pruebas y demostración fueron completamente satisfactorios, cumpliendo con la totalidad de los objetivos y criterios de evaluación establecidos en la práctica. Al energizar el prototipo, el microcontrolador ESP32 inicializó sin inconvenientes los periféricos I2C, 1-Wire y matriciales, presentando el mensaje de bienvenida en la pantalla LCD 16×2 y comenzando de inmediato la adquisición de temperatura dentro de la cámara térmica con actualizaciones estables cada segundo.

Durante la evaluación de la interfaz de usuario, se comprobó que el botón selector conectado al GPIO 25 alternó de manera limpia y sin rebotes el cursor asterisco * entre los parámetros Min y Max. Al introducir distintos valores numéricos mediante el teclado matricial 4×4, tal como ajustar el límite mínimo a 28 °C y modificar el límite máximo, la pantalla reflejó en tiempo real los dígitos ingresados y actualizó los umbrales de control al presionar la tecla #, rechazando además configuraciones incoherentes donde el límite mínimo intentara superar al máximo.

Asimismo, se verificó físicamente la acción de control ON-OFF con histéresis: al encontrarse la temperatura por debajo del umbral superior, el relevador correspondiente al foco calefactor se activó provocando un incremento progresivo de la temperatura interna; una vez alcanzado el límite máximo configurado, el sistema desactivó el calefactor y energizó automáticamente el segundo relevador para encender la celda Peltier y su ventilador, cambiando el indicador en pantalla a [ENF] y reduciendo la temperatura del recinto hasta alcanzar el límite inferior, dentro del cual la banda de histéresis evitó conmutaciones erráticas o oscilaciones rápidas en los contactos mecánicos de los relevadores.

Conclusiones

La integración de hardware embebido con programación estructurada y no bloqueante permitió construir un controlador térmico ON-OFF funcional, seguro y reconfigurable en tiempo real, consolidando los conocimientos teóricos y prácticos sobre adquisición de señales digitales, buses de comunicación serie, etapas de potencia aisladas y sistemas de control automático. Se comprobó que el uso del sensor digital DS18B20 bajo el protocolo 1-Wire ofrece mediciones inmunes al ruido eléctrico generado por la conmutación de cargas inductivas y resistivas cercanas, mientras que el acoplamiento de un adaptador I2C a la pantalla LCD optimiza significativamente la disponibilidad de pines GPIO en el ESP32 para integrar interfaces matriciales complejas.

De igual manera, la implementación de una banda de histéresis configurable demostró ser indispensable para proteger la integridad electromecánica de los relevadores y la vida útil de la celda Peltier, evitando ciclos de encendido y apagado continuos en la frontera de referencia. El cumplimiento exitoso de todas las pruebas funcionales frente al docente valida tanto el diseño electrónico del recinto térmico como la lógica de software desarrollada en PlatformIO.

Referencias

Analog Devices. *DS18B20: Programmable Resolution 1-Wire Digital Thermometer Datasheet*. Analog Devices, Inc., 2019.

Banzi, M., & Shiloh, M. *Getting Started with Arduino*. Maker Media, Inc., 2022.

Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems Co., Ltd., 2023.

Monk, S. *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education, 2022.

NXP Semiconductors. *PCF8574; PCF8574A: Remote 8-bit I/O Expander for I2C-bus with Interrupt Product Data Sheet*. NXP B.V., 2013.

Ogata, K. *Ingeniería de control moderna*. Pearson Educación, 2010.

PlatformIO. *PlatformIO Documentation: Espressif 32 Platform and Arduino Framework*. PlatformIO Labs, 2024.